3.3 Sprint 6 : Téléconsultation & Paiement en Ligne, Gestion des profils famille et administration

3.3.1 Raffinement de Sprint 6

Raffinement du cas « Effectuer une téléconsultation »

La figure 3.14 présente la décomposition du cas « Effectuer une téléconsultation ». Le patient et le médecin rejoignent une salle vidéo (Jitsi Meet) intégrée à la plateforme. Le médecin peut accéder au dossier patient en parallèle.

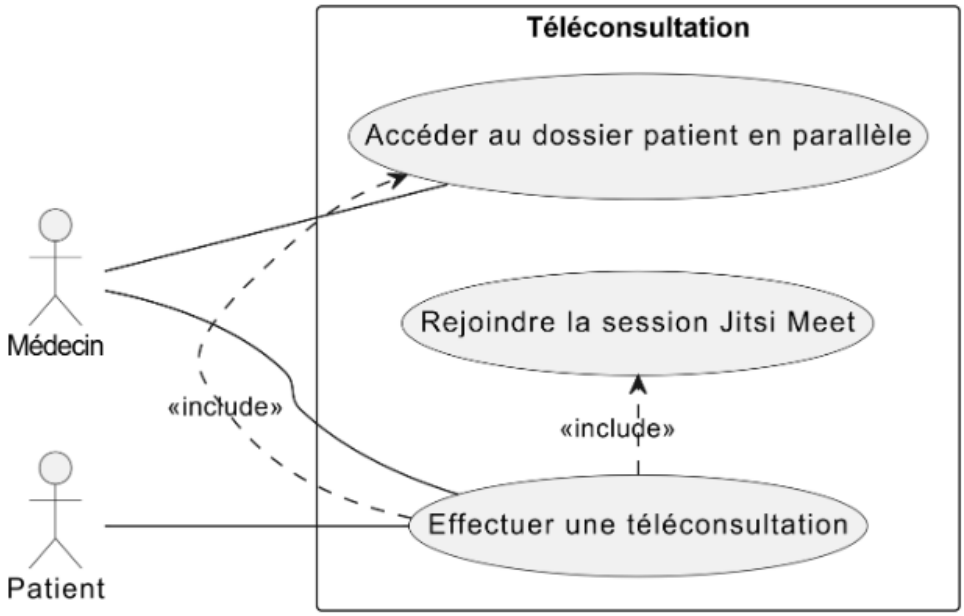


FIGURE 3.14 – Diagramme de cas d'utilisation – Téléconsultation

Le diagramme de classe analytique (figure 3.15) représente les entités impliquées dans la téléconsultation, centré sur l'entité *appointments* avec *consultation_type = TELECONSULTATION* et *video_room_id* (identifiant de salle Jitsi Meet), reliant *users* (Médecin et Patient) à l'entité *patients* pour l'accès au dossier médical en parallèle.

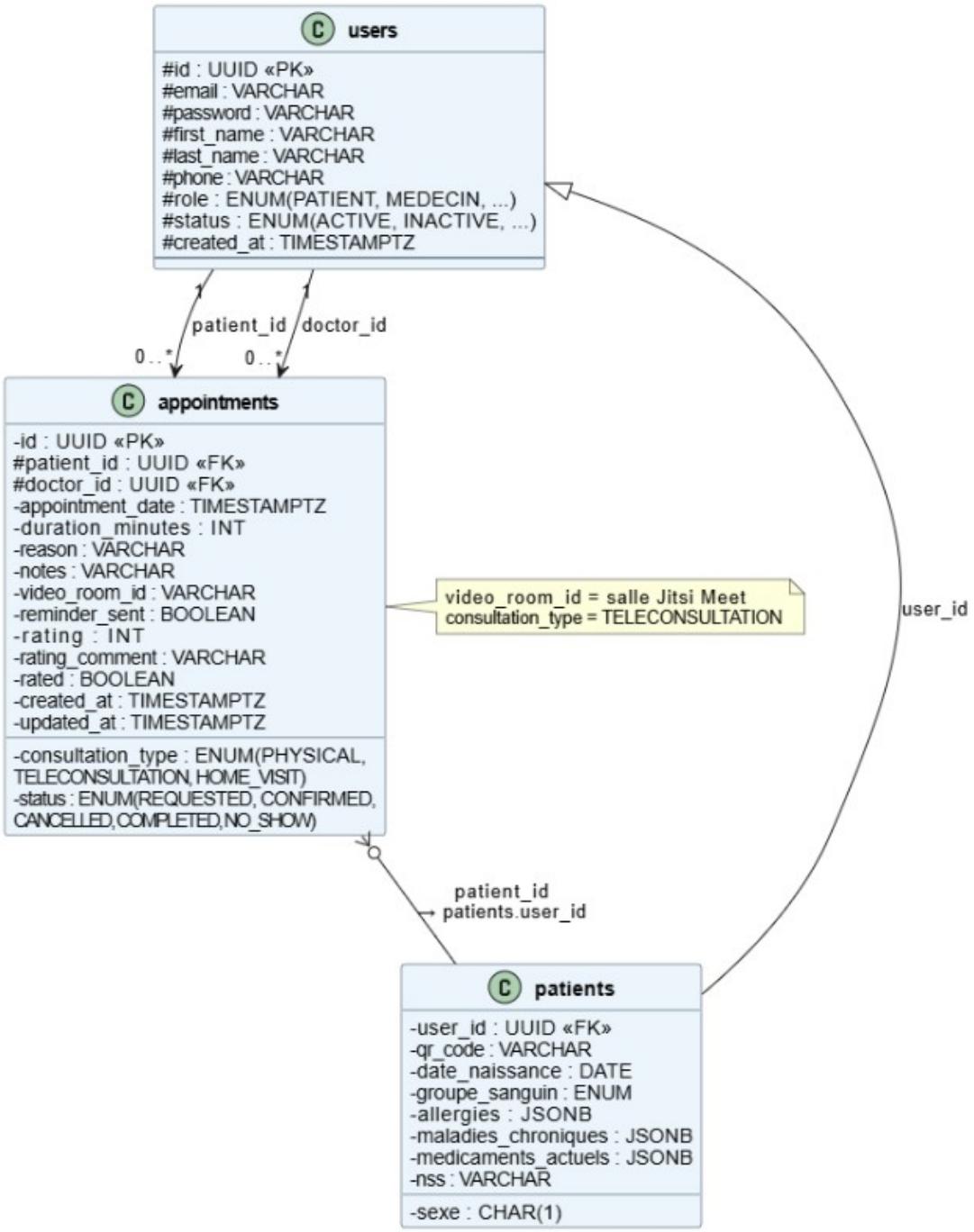


FIGURE 3.15 – Diagramme de classe Analytique – Téléconsultation

Le diagramme de séquence analytique (figure 3.16) détaille quatre flux entre le *Patient*, le *Médecin* et le *Système* : ouverture de la salle vidéo côté médecin (POST /appointments/ :id/video-room, UPDATE video_room_id = 'mediconnect- :uuid', URL Jitsi construite côté frontend), ouverture côté patient (même URL Jitsi Meet), accès au dossier patient en parallèle (GET /dossier-medical/ :patientId avec vérification des permissions, SELECT FROM dossier_acces_medecin, dossier filtré selon permissions ou 403 Accès non autorisé), et fin de session (PATCH /appointments/ :id/complete, UPDATE SET status='COMPLETED', notes).

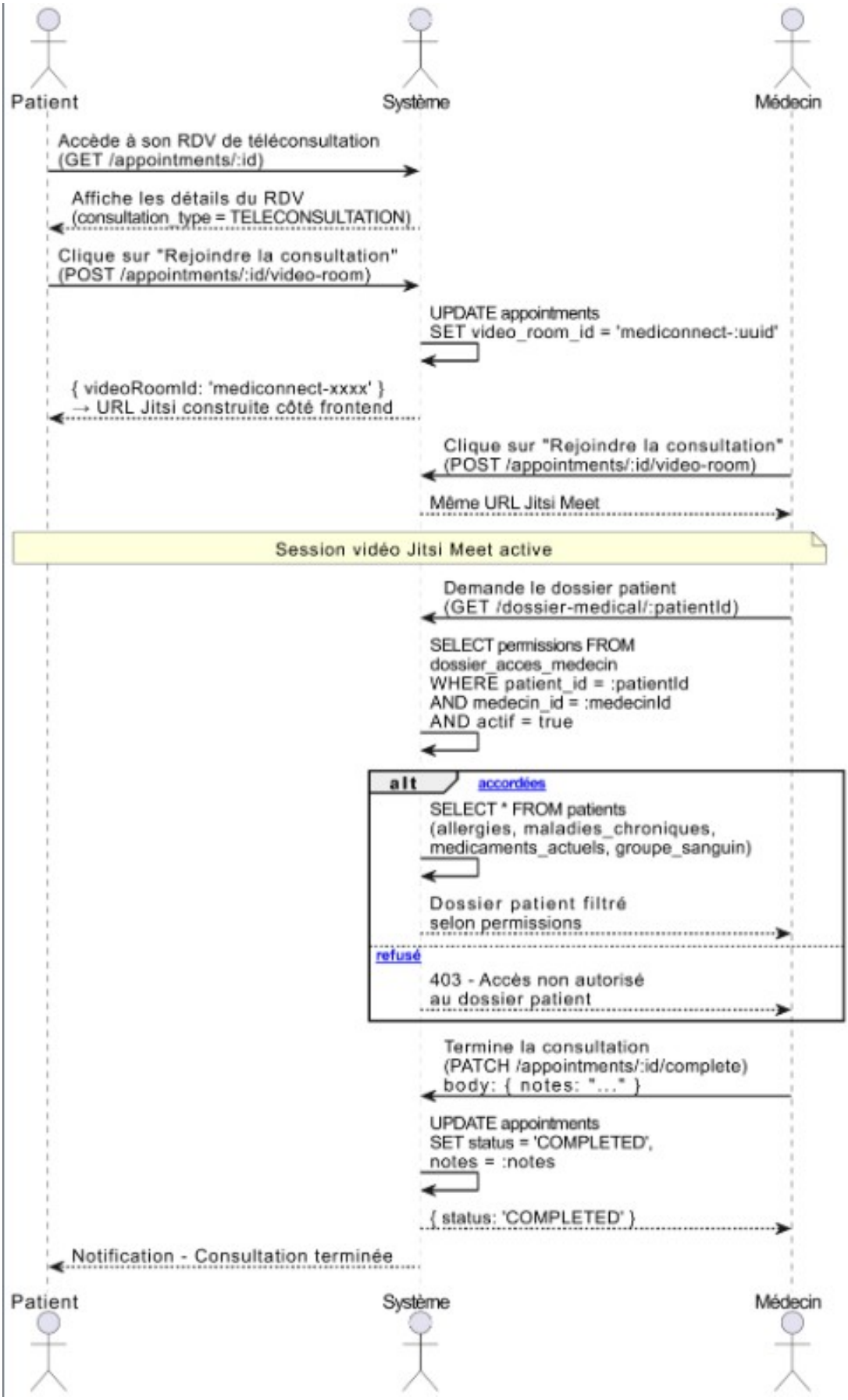


FIGURE 3.16 – Diagramme de séquence Analytique – Téléconsultation

TABLE 3.4 – Description du cas d'utilisation « Effectuer une téléconsultation »

Cas	Effectuer une téléconsultation
Acteur	Médecin, Patient
Condition préalable	Le rendez-vous est confirmé avec le type `TELECONSULTATION`, et les deux acteurs sont authentifiés sur la plateforme MediConnect.
Condition résultante	La session vidéo Jitsi Meet est ouverte, le médecin a accédé au dossier patient selon ses permissions, et la consultation est clôturée avec le statut `COMPLETED`.
Description du scé-nario	— Le patient accède à son rendez-vous de téléconsultation et clique sur « Rejoindre la consultation » ; le système génère un identifiant de salle Jitsi Meet (`video_room_id`). — Le médecin clique sur « Rejoindre la consultation » et accède à la même URL Jitsi Meet ; la session vidéo est active pour les deux parties. — Le médecin accède au dossier patient en parallèle ; le système vérifie les permissions et filtre les données selon le périmètre autorisé. — Le médecin termine la consultation ; le système met à jour le statut du rendez-vous à `COMPLETED` et enregistre les notes cliniques.
Scénarii alternatifs	Accès au dossier non autorisé : le système retourne une erreur 403 (Accès non autorisé au dossier patient). Problème de connexion vidéo : le patient ou le médecin ne peut pas rejoindre la salle Jitsi Meet ; le système affiche un message d'erreur et invite à réessayer.

Raffinement du cas « Payer une consultation en ligne »

La figure 3.17 illustre la décomposition du cas « Payer une consultation en ligne ». Le patient règle la consultation via Flouci, avec *Confirmer le rendez-vous* comme sous-cas inclus et *Paiement via Flouci* comme extension.

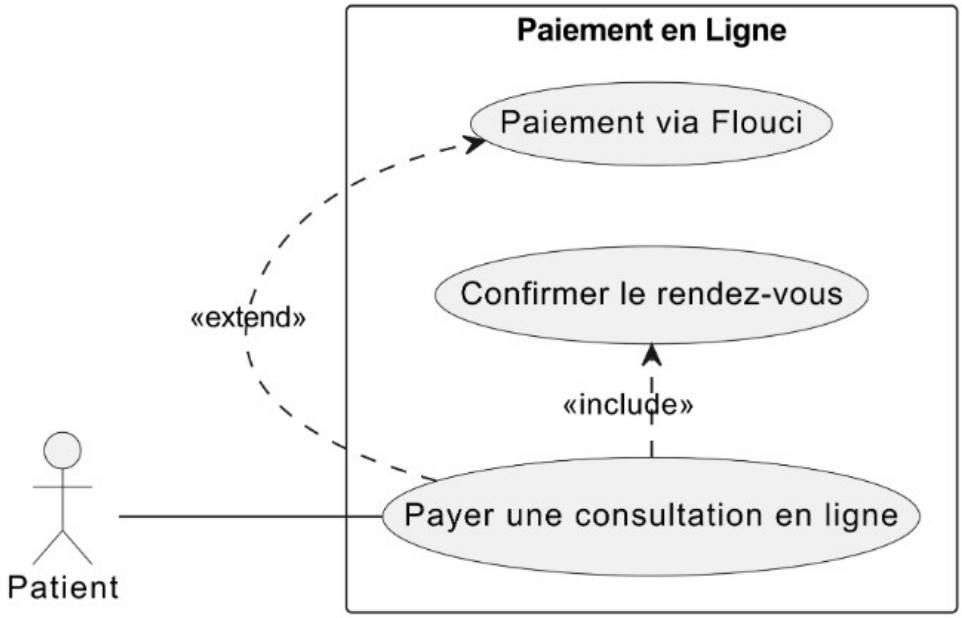


FIGURE 3.17 – Diagramme de cas d'utilisation – Paiement en ligne

Le diagramme de classe analytique (figure 3.18) représente les entités du module de paiement, centré sur *appointments* redirigé vers l'*API Flouci* (externe) avec les méthodes *processPayment()*, *verifyPayment()* et *generateInvoice()*. Aucune table de paiement dédiée n'est nécessaire : le statut de l'appointment passe à CONFIRMED après validation Flouci.

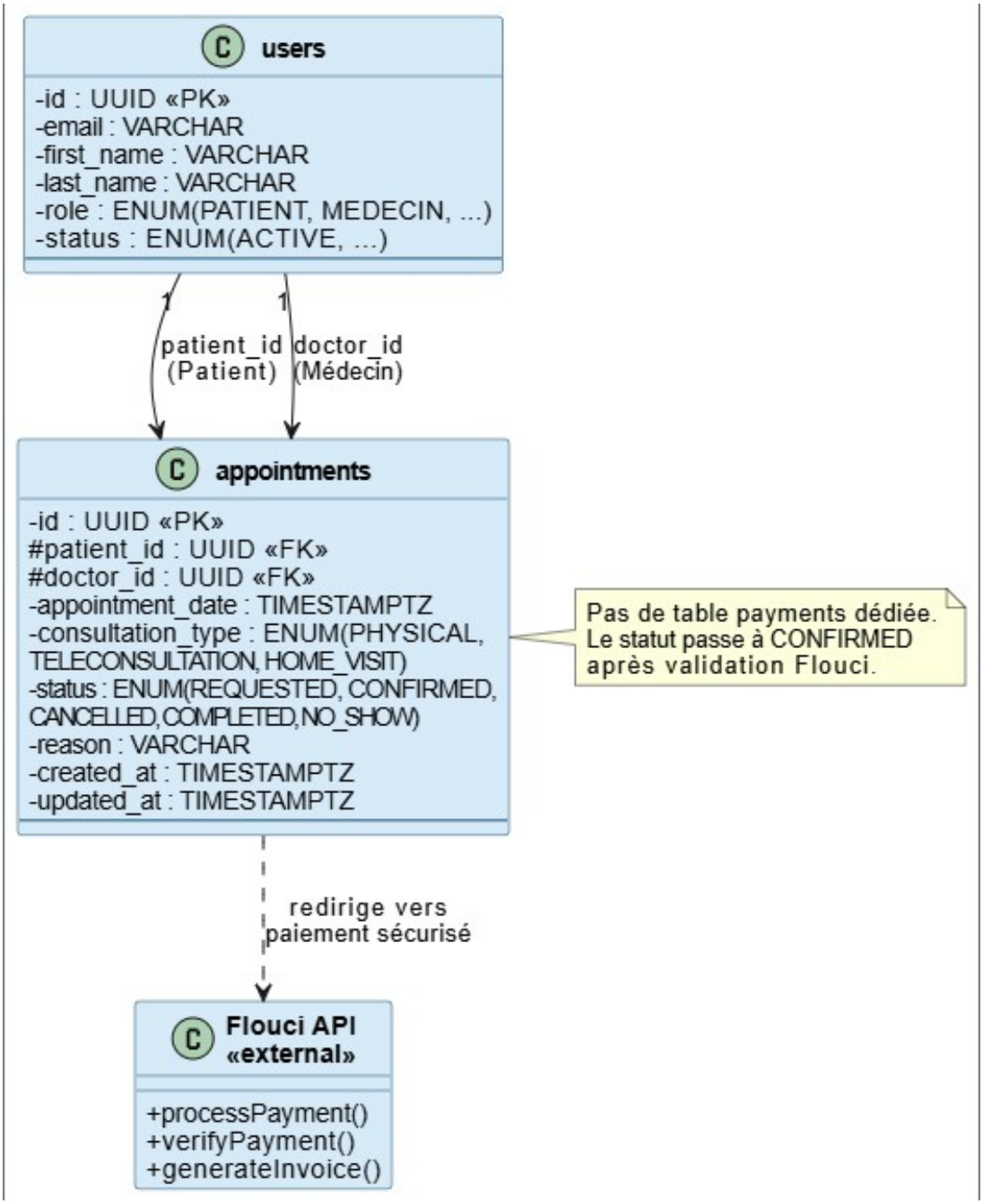


FIGURE 3.18 – Diagramme de classe Analytique – Paiement en ligne

Le diagramme de séquence analytique (figure 3.19) détaille le flux entre le *Patient*, le *Système* et l'*API Flouci* : consultation de l'historique des paiements (GET /payments/history, SELECT FROM appointments JOIN users ORDER BY appointment_date DESC), initiation du paiement (POST /payments/checkout/ :appointmentId, création session Flouci), redirection vers la page Flouci, vérification du paiement (GET /payments/verify), puis deux scénarios alternatifs : paiement confirmé (status : SUCCESS → UPDATE appointments SET status=‘CONFIRMED‘) ou échoué (status : FAILED, invitation à réessayer), et erreur de connexion Flouci (service indisponible).

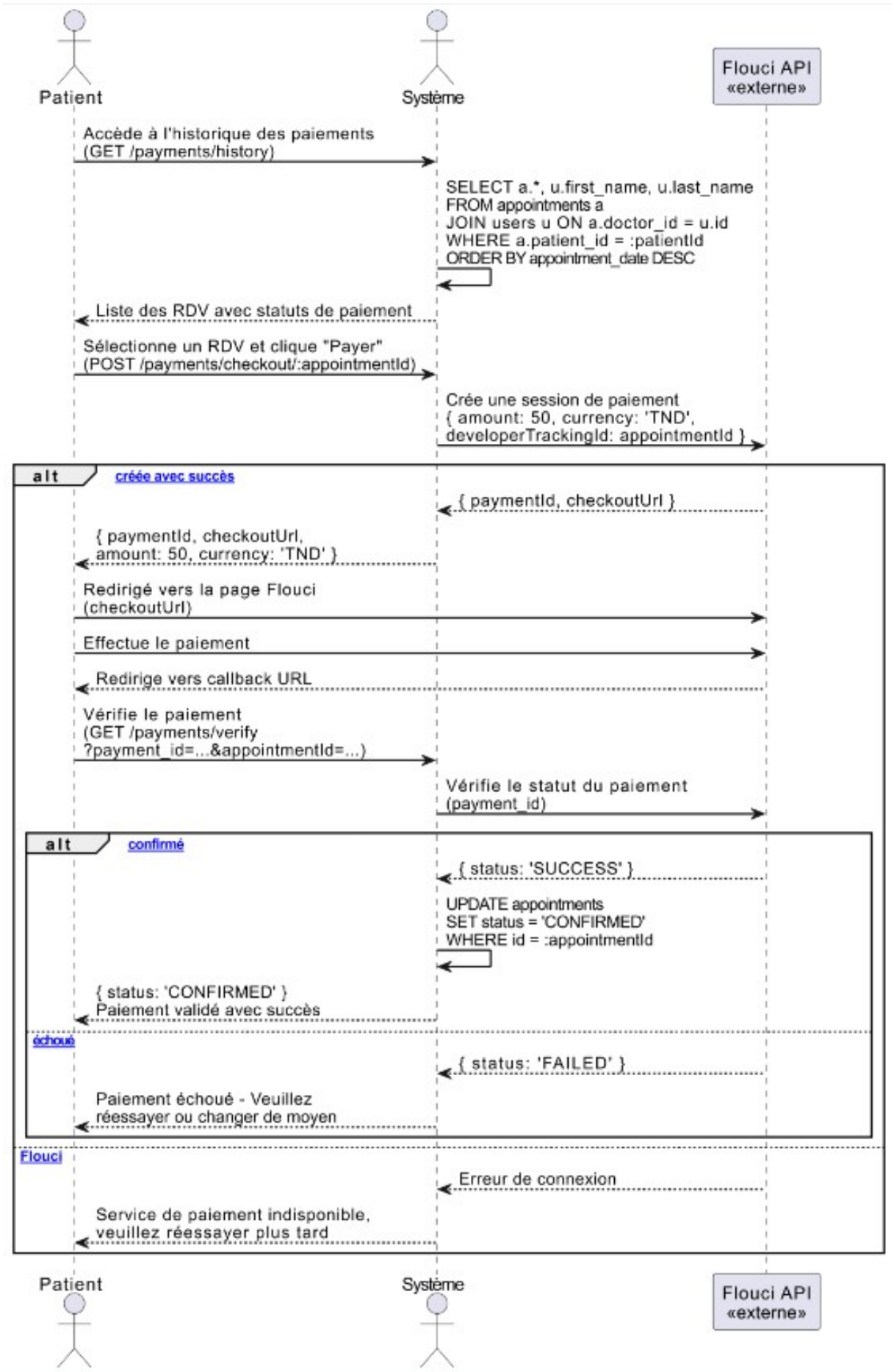


FIGURE 3.19 – Diagramme de séquence Analytique – Paiement en ligne

TABLE 3.5 – Description du cas d'utilisation « Payer une consultation en ligne »

Cas	Payer une consultation en ligne
Acteur	Patient
Condition préalable	Le patient est authentifié et dispose d'un rendez-vous en attente de paiement sur la plateforme MediConnect.
Condition résultante	Le paiement est validé via l'API Flouci et le statut du rendez-vous est mis à jour à `CONFIRMED`.
Description du scé-nario	— Le patient accède à son historique des paiements et consulte la liste de ses rendez-vous avec leurs statuts. — Le patient sélectionne un rendez-vous et clique sur « Payer » ; le système crée une session de paiement Flouci et retourne un `checkoutUrl`. — Le patient est redirigé vers la page Flouci, effectue le paiement, puis est redirigé vers la plateforme via le callback URL. — Le système vérifie le statut du paiement auprès de l'API Flouci ; en cas de succès, le statut du rendez-vous passe à `CONFIRMED` et un message de confirmation est affiché.
Scénarii alternatifs	Paiement échoué : l'API Flouci retourne `FAILED` ; le système invite le patient à réessayer ou à changer de moyen de paiement. Erreur de connexion Flouci : le service est indisponible ; le système affiche un message d'indisponibilité et invite à réessayer ultérieurement.

Raffinement du cas « Gérer les profils famille »

La figure 3.20 montre la décomposition du cas « Gérer les profils famille ». Le patient principal peut ajouter (avec extension *saisir dossier médical membre*), modifier, supprimer (avec extension *consulter dossier médical membre*), consulter les profils famille et prendre RDV pour un membre, avec notification systématique.

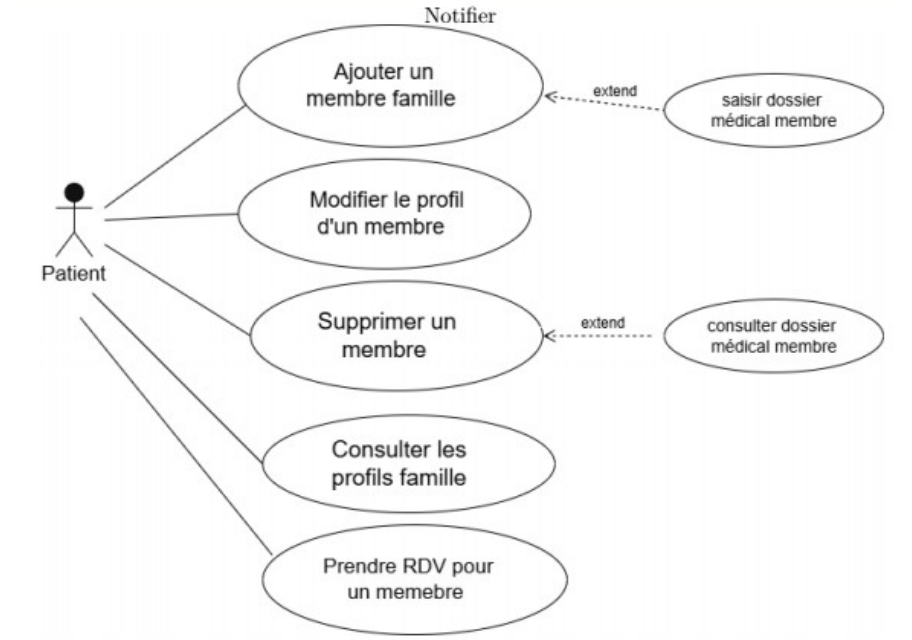


FIGURE 3.20 – Diagramme de cas d'utilisation – Gestion des profils famille

Le diagramme de classe analytique (figure 3.21) représente les entités du module familial, centré sur *family_members* (relation ENFANT, CONJOINT, PARENT, AUTRE avec CASCADE DELETE) lié à *patients* via *patient_id*, et à *appointments* via *beneficiary_id* pour la prise de RDV au nom d'un membre.

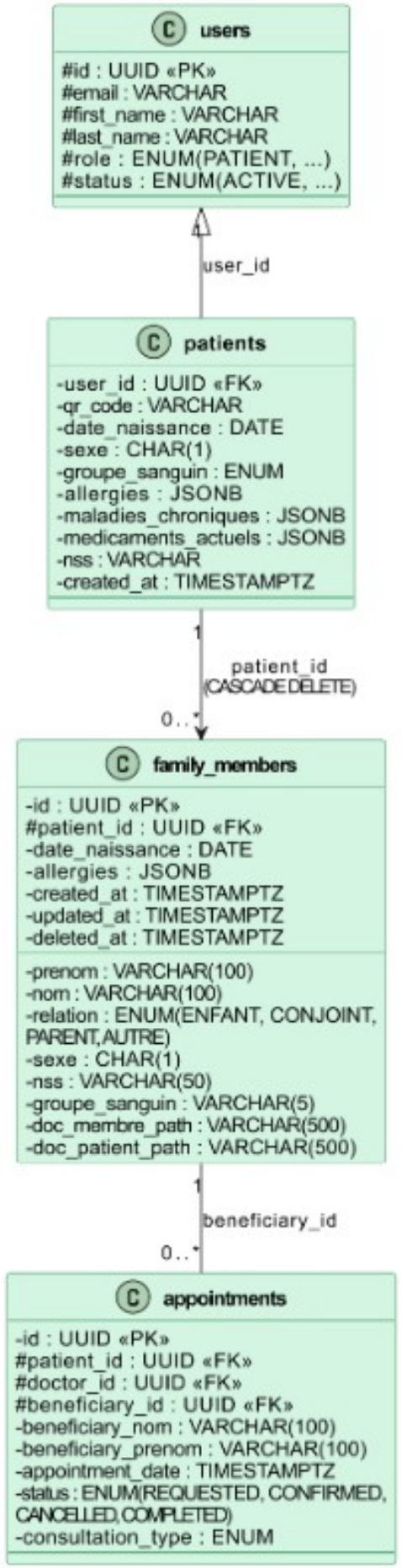


FIGURE 3.21 – Diagramme de classe Analytique – Gestion des profils famille

Le diagramme de séquence analytique (figure 3.22) détaille quatre flux entre le *Patient* et le *Système* : ajout d'un membre (POST /family-members avec INSERT INTO family_members, 201 si valide ou 400 si données invalides), modification (PATCH /family-members/ :id, vérification appartenance, UPDATE ou 403 Accès refusé), suppression (DELETE /family-members/ :id, soft delete : UPDATE SET deleted_at = NOW() ou 404 Membre introuvable), et prise de RDV pour un membre (POST /appointments + beneficiaryId, INSERT INTO appointments avec status=‘REQUESTED‘).

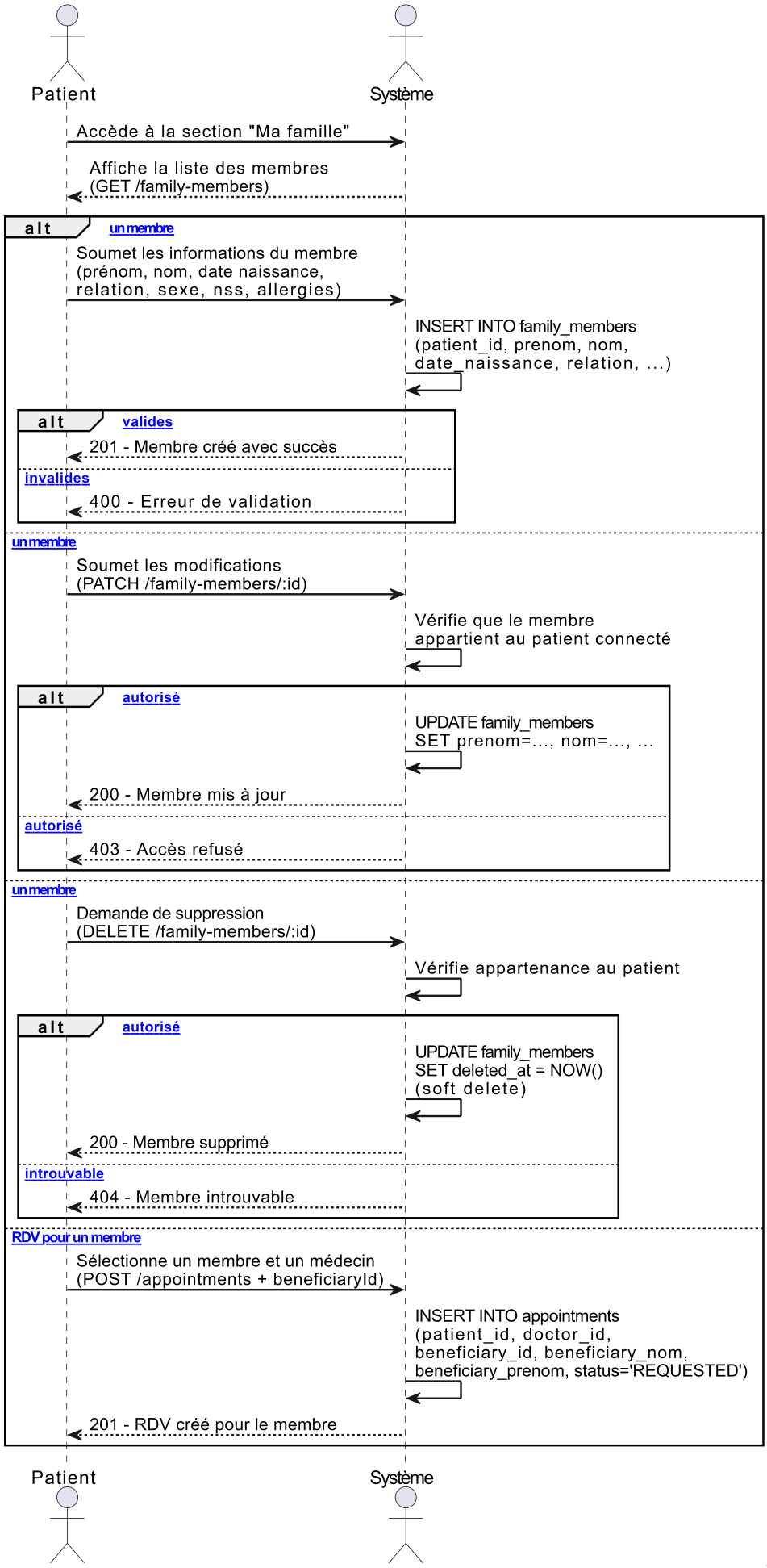


FIGURE 3.22 – Diagramme de séquence Analytique – Gestion des profils famille

TABLE 3.6 – Description du cas d'utilisation « Gérer les profils famille »

Cas	Gérer les profils famille
Acteur	Patient
Condition préalable	Le patient est authentifié sur la plateforme MediConnect et possède un compte actif.
Condition résultante	Les profils des membres de la famille sont créés, modifiés ou supprimés ; des rendez-vous peuvent être pris au nom d'un membre.
Description du scé-nario	— Le patient accède à la section « Ma famille » et consulte la liste des membres enregistrés. — Le patient peut ajouter un membre (prénom, nom, date de naissance, relation, sexe, NSS, allergies) ; le système insère l'entrée dans `family_members` et retourne un code 201. — Le patient peut modifier les informations d'un membre existant ; le système vérifie l'appartenance au patient avant d'effectuer la mise à jour. — Le patient peut supprimer un membre ; le système effectue un soft delete (`deleted_at = NOW()`). — Le patient peut prendre un rendez-vous au nom d'un membre en sélectionnant le membre et un médecin ; le système crée le rendez-vous avec le statut `REQUESTED`.
Scénarii alternatifs	Données invalides lors de l'ajout : le système retourne une erreur 400. Modification non autorisée : le membre n'appartient pas au patient connecté ; le système retourne une erreur 403 (Accès refusé). Suppression d'un membre introuvable : le système retourne une erreur 404 (Membre introuvable).

Raffinement du cas « Gérer les comptes utilisateurs »

La figure 3.23 illustre la décomposition du cas « Gérer les comptes utilisateurs ». L'administrateur peut créer, modifier et désactiver les comptes des médecins, patients et pharmaciens, avec *Valider/Rejeter un compte professionnel* étendu par *Notifier l'utilisateur*.

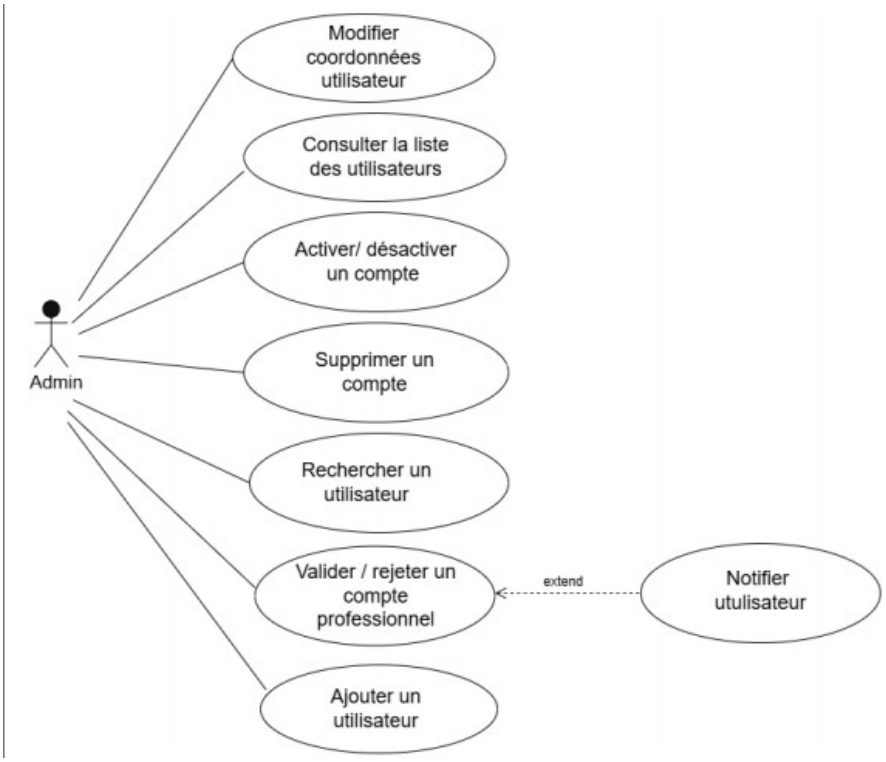


FIGURE 3.23 – Diagramme de cas d'utilisation – Gestion des comptes utilisateurs

Le diagramme de classe analytique (figure 3.24) représente les entités du module d'administration, centré sur l'acteur *Admin* (lui-même un *users* avec role = ADMIN) gérant l'entité *users* (statuts : ACTIVE, INACTIVE, SUSPENDED, PENDING_VERIFICATION) avec déclenchement automatique de *notifications* (types DOCTOR_APPROVED / DOCTOR_REJECTED) lors de la validation ou du rejet d'un compte professionnel.

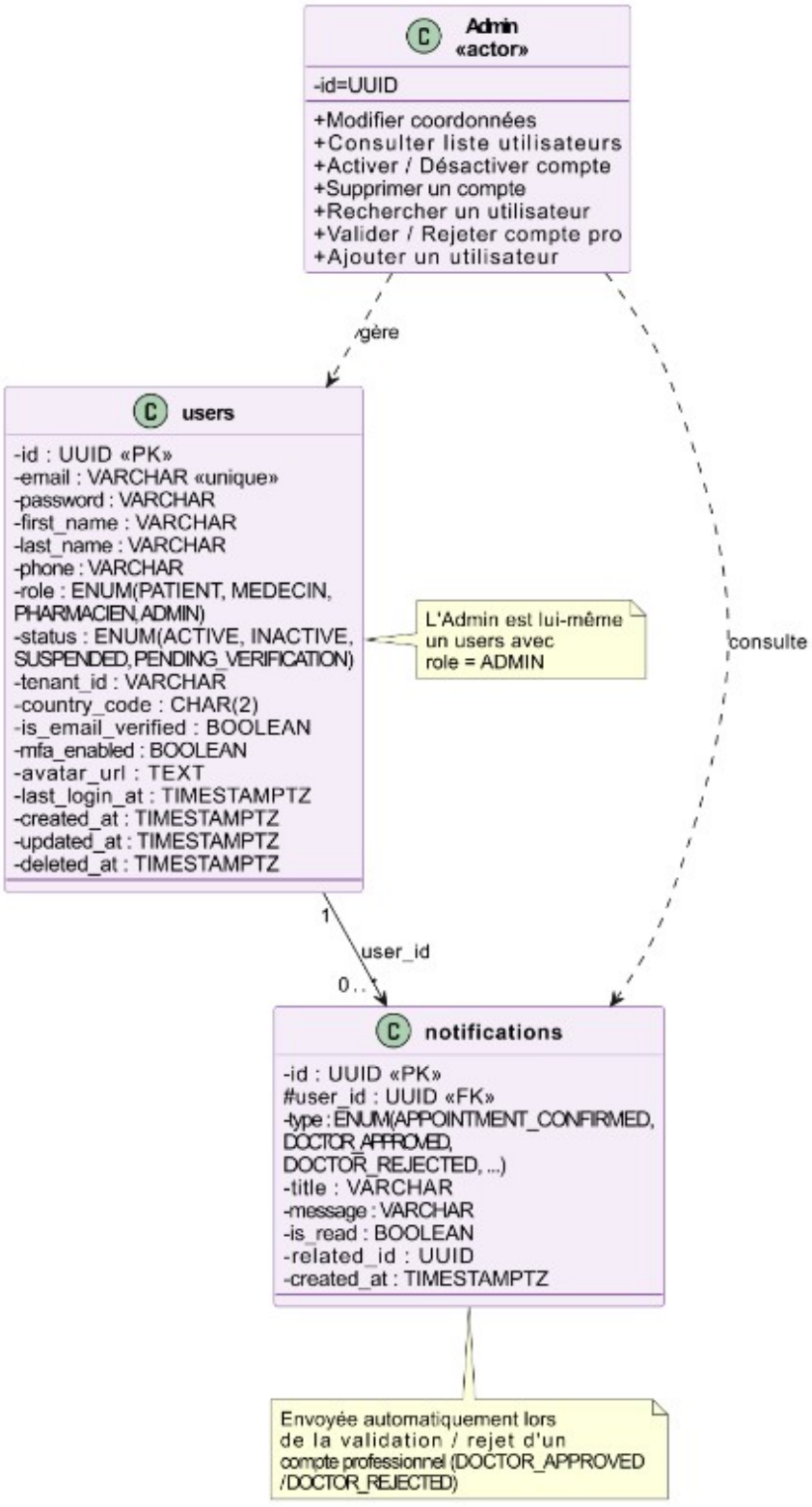


FIGURE 3.24 – Diagramme de classe Analytique – Gestion des comptes utilisateurs

Le diagramme de séquence analytique (figure 3.25) détaille cinq flux entre l'*Administrateur* et le *Système* : ajout d'un utilisateur (informations soumises → Utilisateur créé), modification des coordonnées (Coordonnées mises à jour), activation/désactivation (Statut mis à jour), suppression (Compte supprimé après confirmation), recherche par critère (Liste filtrée), et validation/rejet d'un compte professionnel (PATCH /admin/users/ :id/approve

→ Compte activé + Notification envoyée au professionnel, ou PATCH /admin/users/ :id/reject
→ Compte suspendu + Notification de refus envoyée).

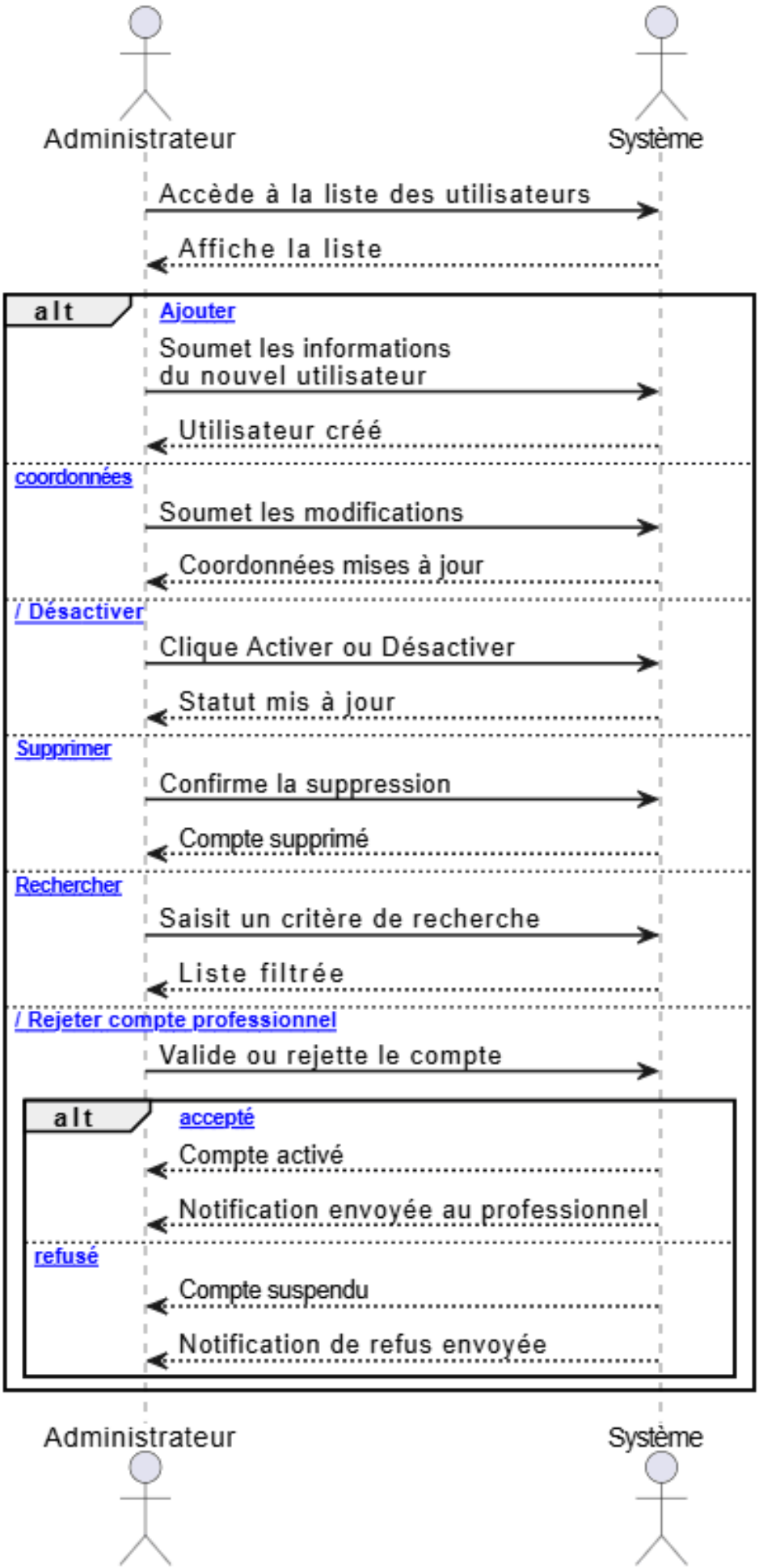


FIGURE 3.25 – Diagramme de séquence Analytique – Gestion des comptes utilisateurs

TABLE 3.7 – Description du cas d'utilisation « Gérer les comptes utilisateurs »

Cas	Gérer les comptes utilisateurs
Acteur	Administrateur
Condition préalable	L'administrateur est authentifié avec le rôle `ADMIN` sur la plateforme MediConnect.
Condition résultante	Les comptes utilisateurs sont créés, modifiés, activés/désactivés ou supprimés ; les comptes professionnels sont validés ou rejetés avec notification automatique.
Description du scé-nario	— L'administrateur accède à la liste des utilisateurs et peut la filtrer par rôle ou statut. — L'administrateur peut ajouter un utilisateur en soumettant ses informations ; le système crée le compte et retourne une confirmation. — L'administrateur peut modifier les coordonnées d'un utilisateur existant ou activer/désactiver son compte. — L'administrateur peut supprimer un compte après confirmation. — L'administrateur consulte les comptes en attente de vérification (`PENDING_VERIFICATION`) et peut valider un compte professionnel : le système passe le statut à `ACTIVE` et envoie une notification `DOCTOR_APPROVED` avec un email de bienvenue. — L'administrateur peut rejeter un compte professionnel : le système passe le statut à `SUSPENDED` et envoie une notification `DOCTOR_REJECTED`.
Scénarii alternatifs	Compte introuvable lors d'une modification ou suppression : le système retourne une erreur 404. Tentative de validation d'un compte déjà actif : le système refuse l'opération et notifie l'administrateur.

3.3.2 Conception de Sprint 6

Conception du cas « Effectuer une téléconsultation »

Le diagramme de classe conceptuel (figure 3.26) présente les classes du module de téléconsultation, structuré en deux couches : le *Frontend Angular* avec les composants *TeleconsultationComponent* et *TeleconsultationPatientComponent* utilisant *AppointmentService* et *DossierService*, et le *Backend NestJS* organisé autour du *C :AppointmentsCon-*

troller (POST /appointments/ :id/video-room, PATCH /appointments/ :id/complete) et *C :DossierMedicalController* (GET /dossier-medical/ :patientId, GET /dossier-medical/ :patientId/acce permissions) avec persistance TypeORM (*Appointment*, *Patient*, *DossierAccesMedecin*, niveaux : LECTURE, LECTURE_ECRITURE, ACCES_COMPLET).

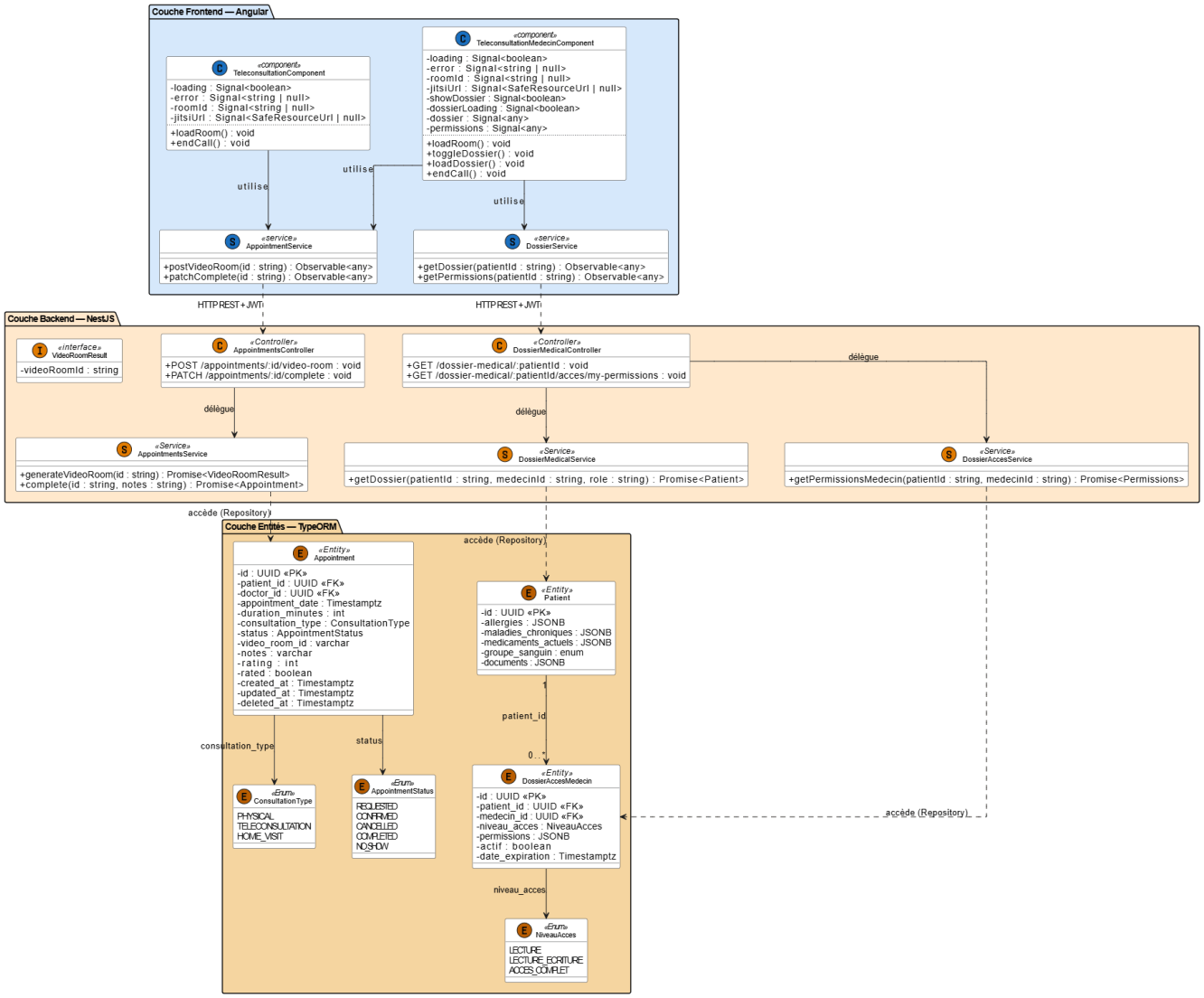


FIGURE 3.26 – Diagramme de classe Conceptuel – Téléconsultation

Le diagramme de séquence conceptuel (figure 3.27) illustre quatre flux : ouverture salle vidéo côté médecin (POST /appointments/ :id/video-room, generateVideoRoom, SELECT FROM appointments, UPDATE SET video_room_id = ‘mediconnect-[uuid]‘), ouverture côté patient (même URL Jitsi construite côté frontend), accès dossier patient en parallèle (GET /dossier-medical/ :patientId avec getDossier + getPermissionsMedecin, SELECT FROM patients JOIN users, SELECT FROM dossier_acces_medecin, dossier filtré selon permissions JSONB), et fin de session (PATCH /appointments/ :id/complete, complete(id, notes), UPDATE SET status=‘COMPLETED‘, notes).

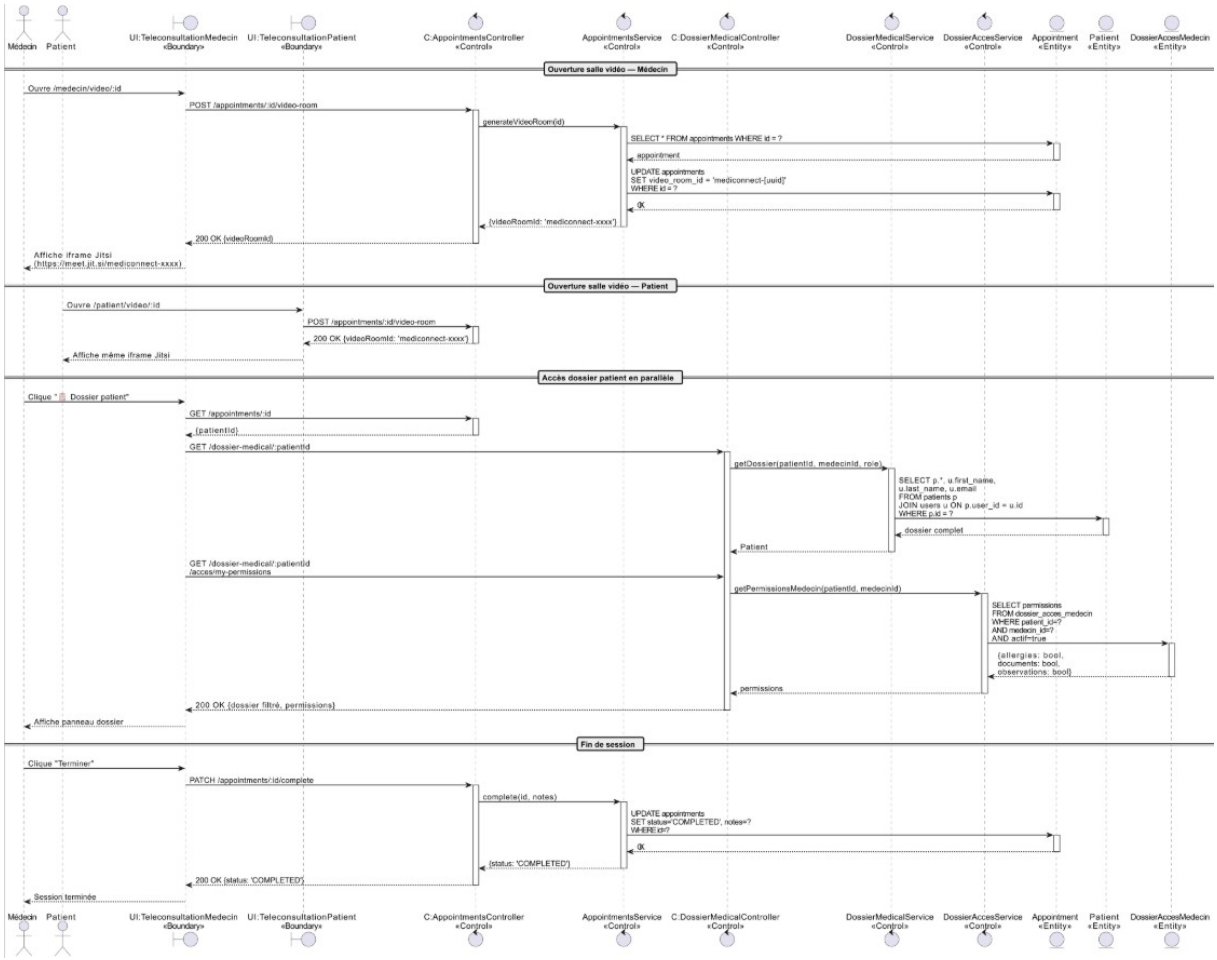


FIGURE 3.27 – Diagramme de séquence Conceptuel – Téléconsultation

Conception du cas « Payer une consultation en ligne »

Le diagramme de classe conceptuel (figure 3.28) présente les classes du module de paiement, structuré en deux couches : le *Frontend Angular* avec le composant *PaymentComponent* et l'*AppointmentServiceFront*, et le *Backend NestJS* organisé autour du *C :PaymentsController* (GET /payments/history, POST /payments/checkout/ :appointmentId, GET /payments/verify), *PaymentsService* et l'API externe *Flouci API* avec persistance TypeORM (*Appointment*, statuts : REQUESTED, CONFIRMED, CANCELLED, COMPLETED, NO_SHOW).

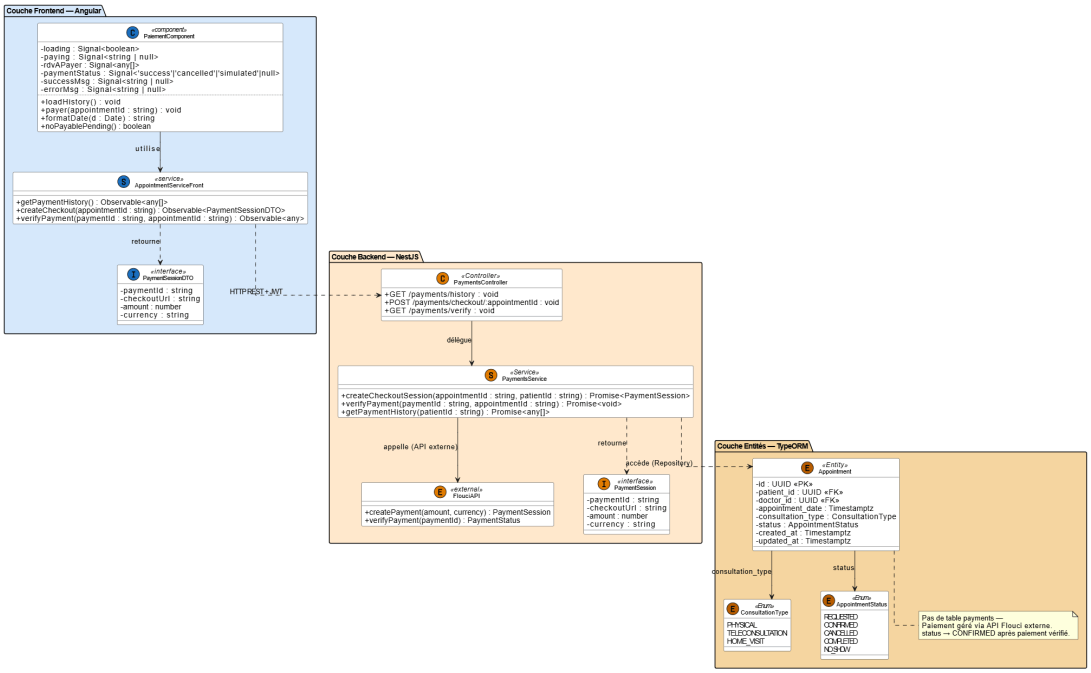


FIGURE 3.28 – Diagramme de classe Conceptuel – Paiement en ligne

Le diagramme de séquence conceptuel (figure 3.29) couvre trois phases : consultation de l'historique (GET /payments/history, getPaymentHistory, SELECT FROM appointments JOIN users ORDER BY appointment_date DESC, 200 OK avec liste RDV + bouton Payer), initiation du paiement (POST /payments/checkout/ :appointmentId, createCheckoutSession, SELECT FROM appointments, POST flouci/generate_payment → paymentId + checkoutUrl → redirection Flouci), et vérification (GET /payments/verify ?payment_id=... &appointmentId=..., verifyPayment, GET flouci/verify_payment/ :paymentId → SUCCESS : UPDATE SET status='CONFIRMED' ou FAILED → invitation à réessayer).

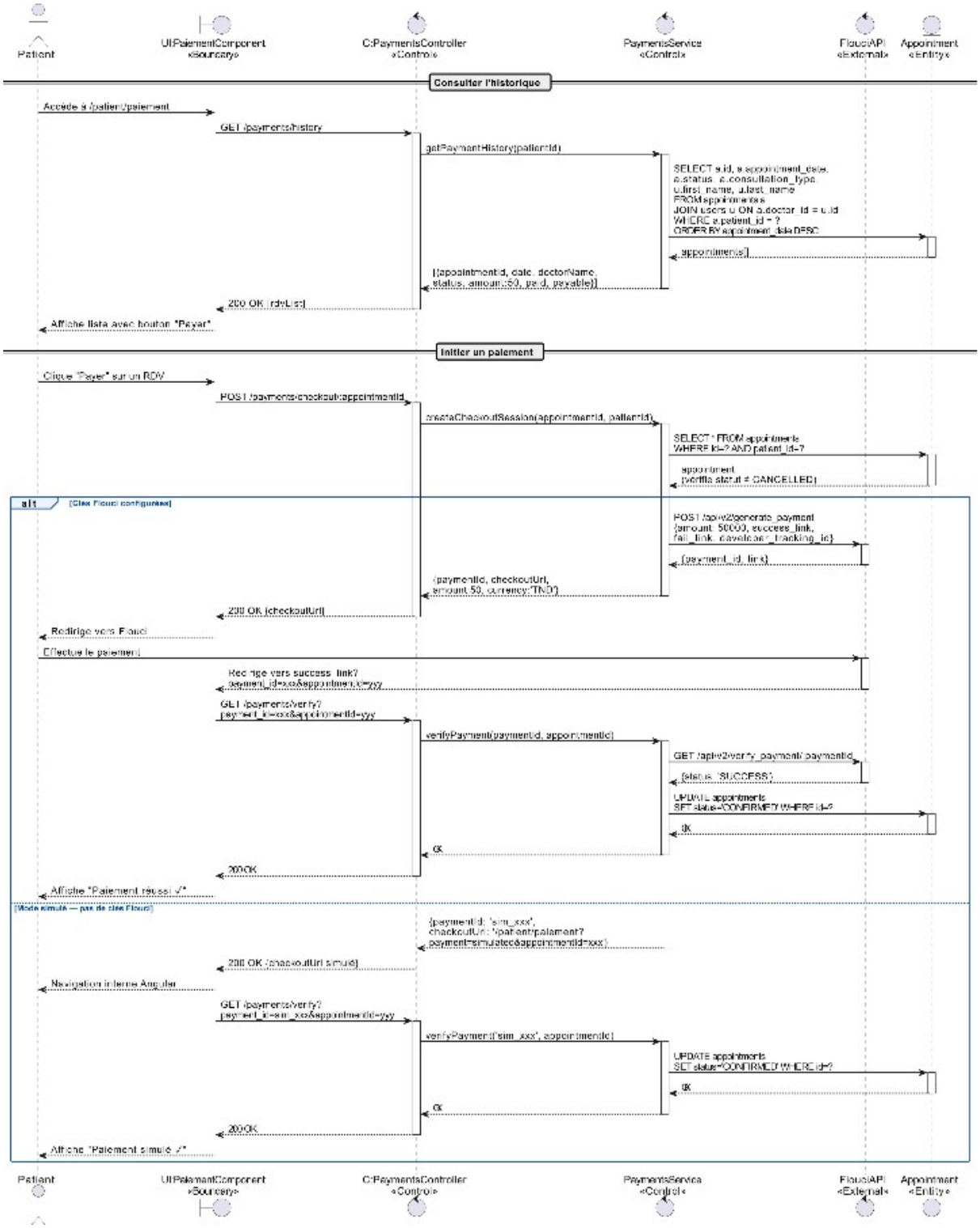


FIGURE 3.29 – Diagramme de séquence Conceptuel – Paiement en ligne

Conception du cas « Gérer les profils famille »

Le diagramme de classe conceptuel (figure 3.30) présente les classes du module familial, structuré en deux couches : le *Frontend Angular* avec le composant *FamilleComponent* et le *FamilyMemberService*, et le *Backend NestJS* organisé autour du *C :FamilyMembersController* (GET, POST, PATCH, DELETE /family-members) et *Fami-*

lyMemberServiceBackend avec persistance TypeORM (*Patient*, *FamilyMember* relation ENFANT/CONJOINT/PARENT/AUTRE, *Appointment* avec beneficiary_id).

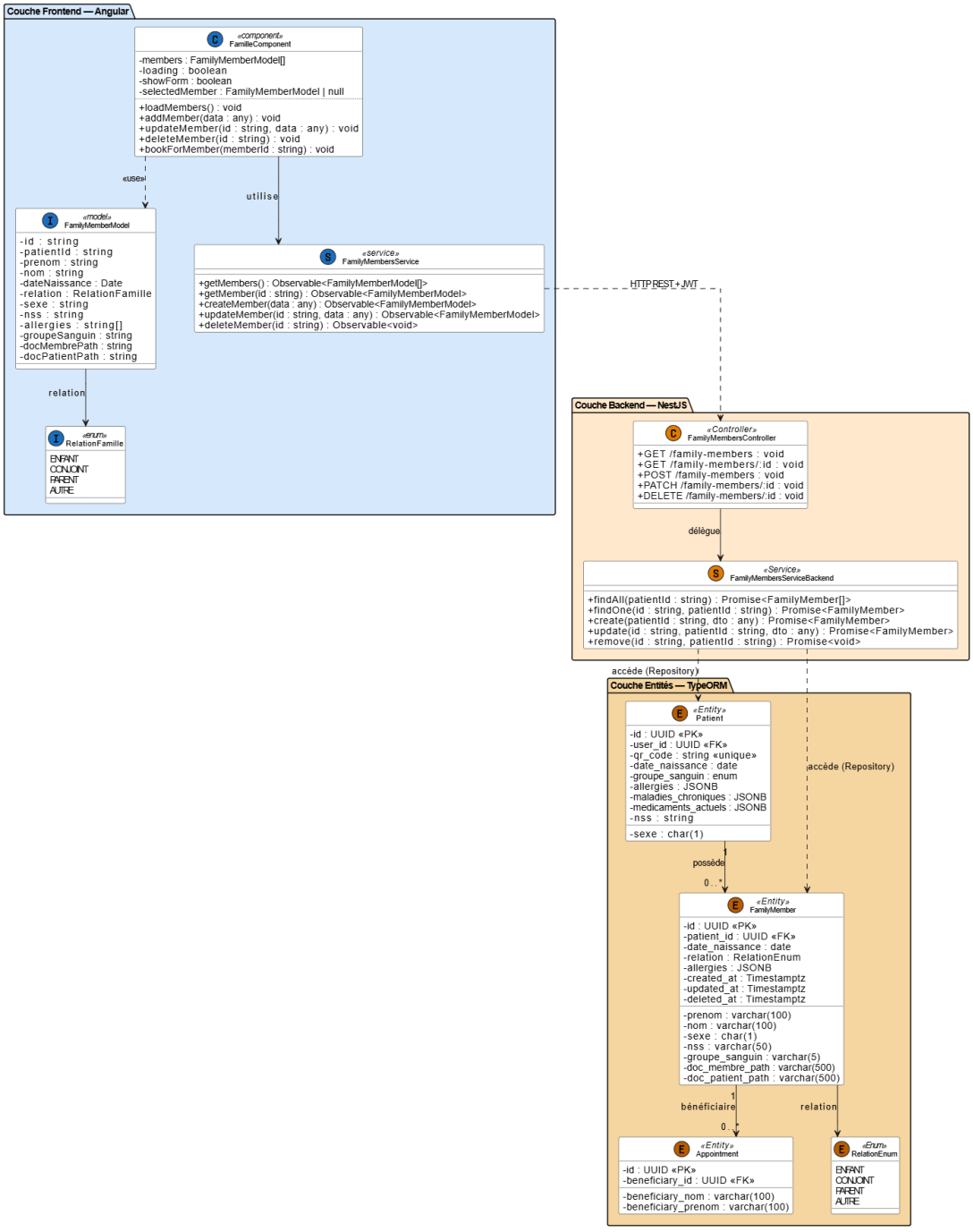


FIGURE 3.30 – Diagramme de classe Conceptuel – Gestion des profils famille

Le diagramme de séquence conceptuel (figure 3.31) illustre trois flux : ajout d'un membre (POST /family-members avec create(patientId, dto), SELECT id FROM patients, INSERT INTO family_members, 201 Created), prise de RDV pour un membre (POST /appointments + beneficiaryId, create(dto), INSERT INTO appointments status='REQUESTED', 201 Created), et suppression (DELETE /family-members/ :id, remove(id, patientId), UPDATE SET deleted_at = NOW(), 200 OK).

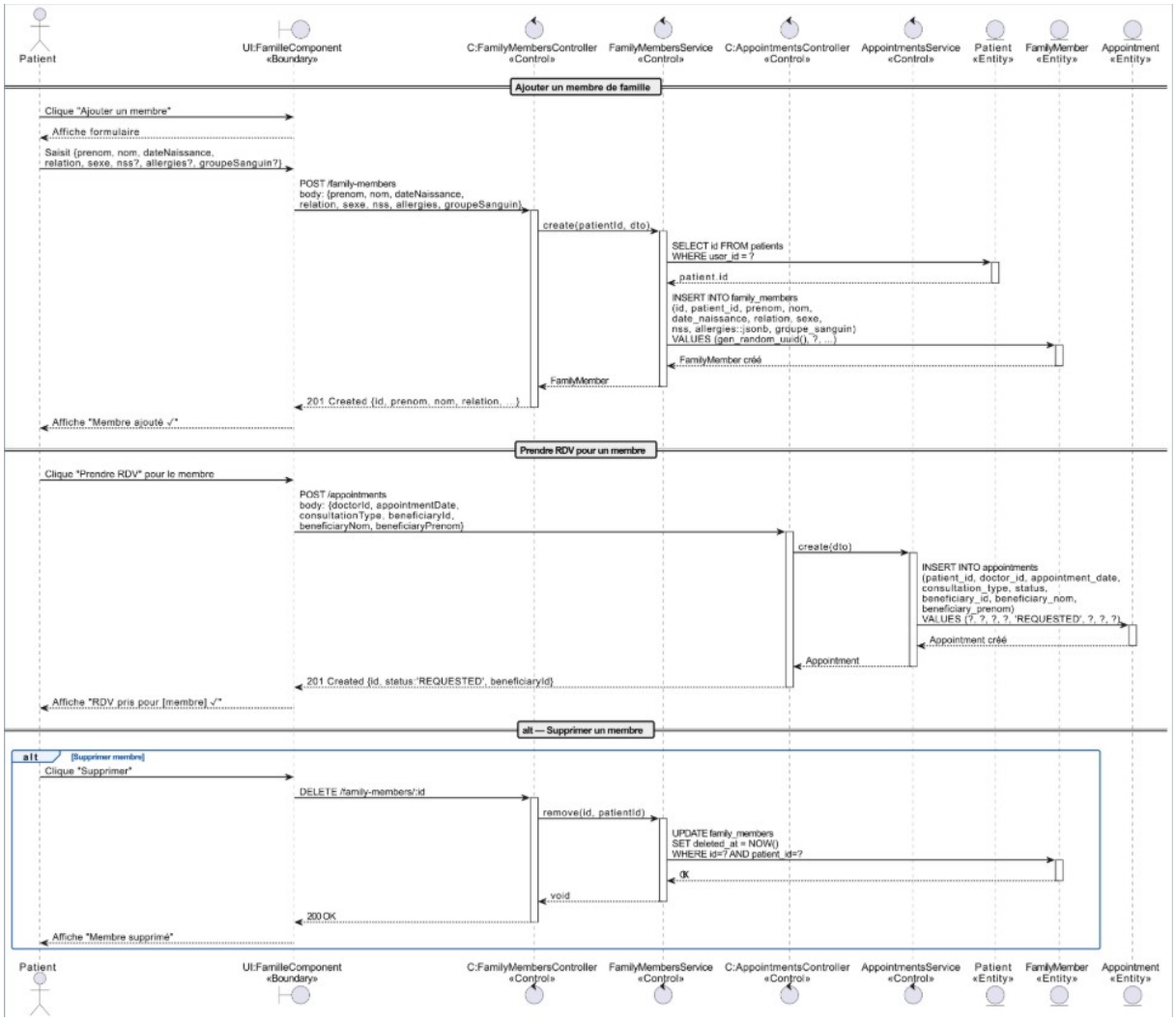


FIGURE 3.31 – Diagramme de séquence Conceptuel – Gestion des profils famille

Conception du cas « Gérer les comptes utilisateurs »

Le diagramme de classe conceptuel (figure 3.32) présente les classes du module d'administration, structuré en deux couches : le *Frontend Angular* avec le composant *AdminDashboardComponent* et l'*AdminUserService*, et le *Backend NestJS* organisé autour du *C :AdminController* (GET /admin/users, GET /admin/users/ :id, PATCH /admin/users/ :id, DELETE /admin/users/ :id, PATCH /admin/users/ :id/approve, PATCH /admin/users/ :id/reject, PATCH /admin/users/ :id/toggle-status), *AdminService* et *NotificationService* avec persistance TypeORM (*User* statuts : ACTIVE, INACTIVE, SUSPENDED, PENDING_VERIFICATION, *Notification* types : DOCTOR_APPROVED, DOCTOR_REJECTED).

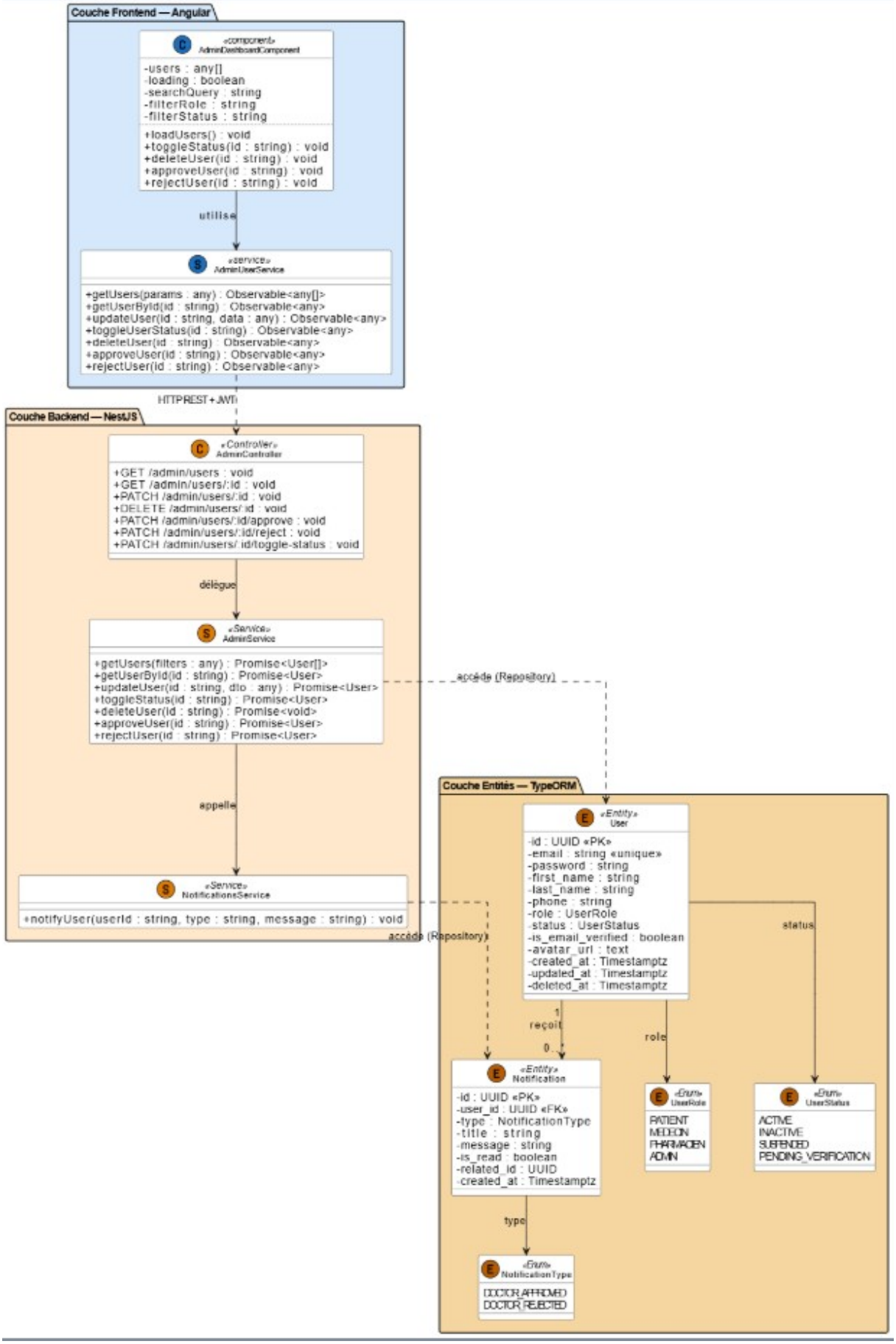


FIGURE 3.32 – Diagramme de classe Conceptuel – Gestion des comptes utilisateurs

Le diagramme de séquence conceptuel (figure 3.33) illustre trois flux entre l'*Utilisateur*, l'*UI :AdminDashboard*, le *C :AdminController*, l'*AdminService* et les entités : consultation des comptes en attente (GET /admin/users ?status=PENDING_VERIFICATION, getUsers(status : 'PENDING_VERIFICATION'), SELECT FROM users WHERE status = PENDING_VERIFICATION, 200 OK liste des comptes), validation d'un compte médecin (PATCH /admin/users/ :id/approve, approveUser(id), UPDATE users SET status='ACTIVE', notifyUser(userId, 'DOCTOR_APPROVED', msg), INSERT INTO notifications, sendWelcomeEmail, 200 OK status : 'ACTIVE'), et rejet d'un compte (PATCH /admin/users/ :id/reject, rejectUser(id), UPDATE users SET status='SUSPENDED', notifyUser(userId, 'DOCTOR_REJECTED', msg), INSERT notifications type=DOCTOR_REJECTED, 200 OK status : 'SUSPENDED').

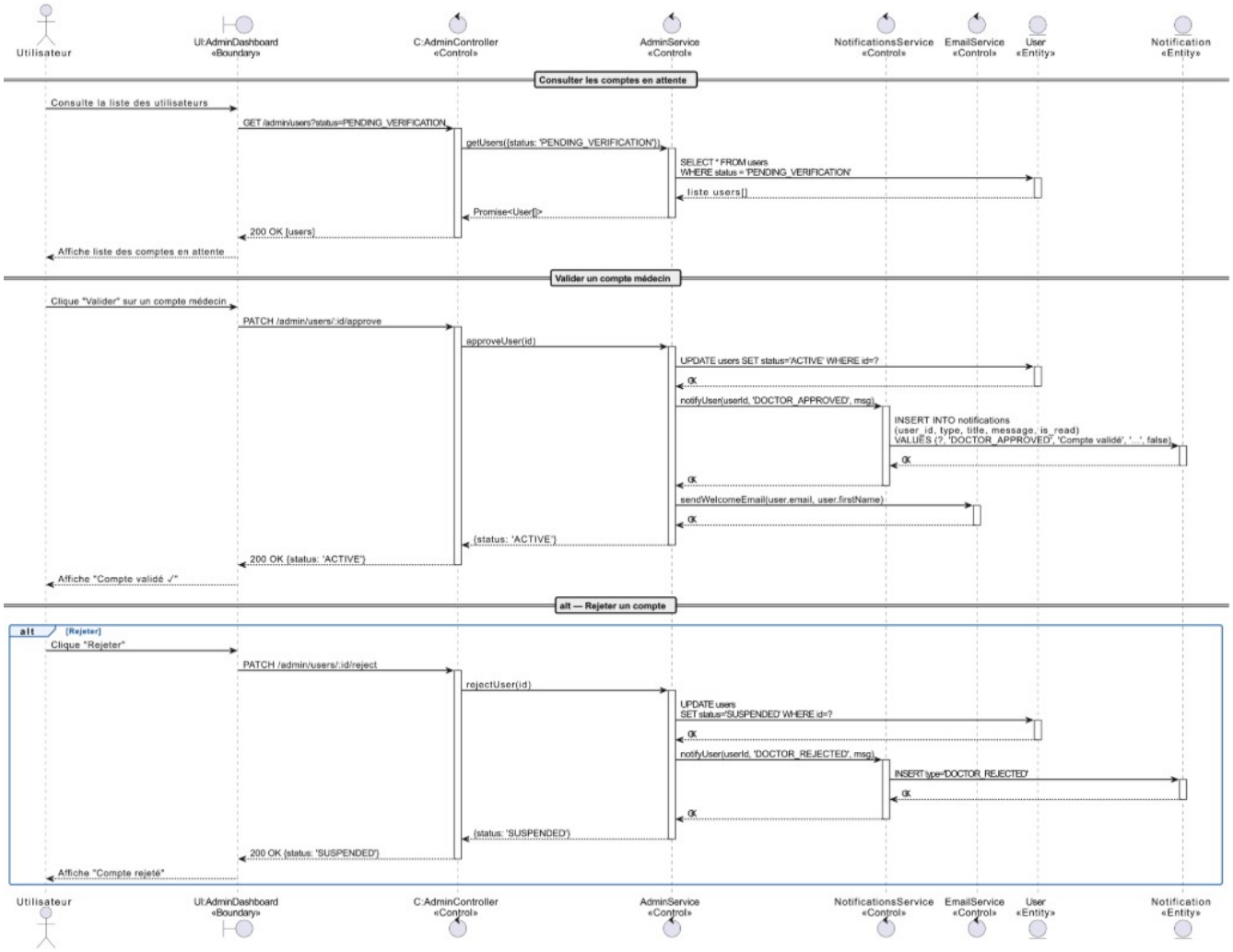


FIGURE 3.33 – Diagramme de séquence Conceptuel – Gestion des comptes utilisateurs

3.3.3 Réalisation de Sprint 6

La figure 3.34 présente l'interface de téléconsultation intégrée avec Jitsi Meet côté patient.



FIGURE 3.34 – Interface de téléconsultation – Vue patient (Jitsi intégré)

La figure 3.35 présente l'interface de paiement en ligne via Flouci.



FIGURE 3.35 – Interface de paiement en ligne (Flouci)

La figure 3.36 présente l'interface de gestion des profils famille.



FIGURE 3.36 – Interface de gestion des profils famille

La figure 3.37 présente l'interface d'administration des comptes utilisateurs.

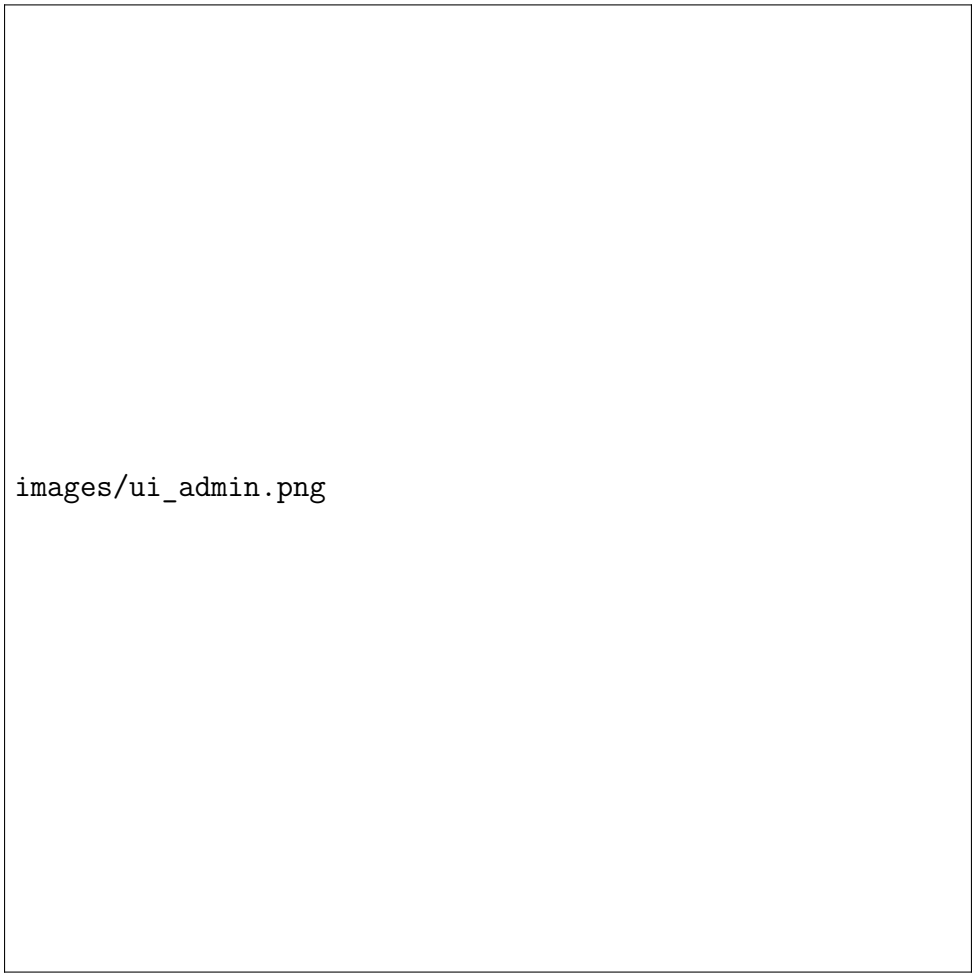


FIGURE 3.37 – Interface d'administration des comptes utilisateurs

Conclusion de la Release 3

La Release 3 a finalisé l'écosystème MediConnect en apportant des fonctionnalités clés de prescription électronique sécurisée, de souveraineté des données patient, de téléconsultation intégrée et de paiement en ligne. L'ensemble du parcours de soin est désormais couvert : de la prise de rendez-vous à la délivrance en pharmacie, en passant par la consultation médicale, l'ordonnance numérique et le partage contrôlé des données. L'architecture technique (Angular 17, NestJS 10, PostgreSQL) a démontré sa scalabilité et sa fiabilité. Les releases futures pourront intégrer l'intelligence artificielle médicale et l'interopérabilité avec les systèmes hospitaliers nationaux.